

PROJECT PROPOSAL

Smart Hotel Management System

Subject: Object-Oriented Programming (OOP)

Language: Java

Date: May 24, 2026

Cyber Security 25



1. Introduction

The Smart Hotel Management System is a Java-based console application designed to automate the core operations of a hotel. The system enables front-desk staff to manage guest check-ins, check-outs, room bookings, and billing — all through a simple and interactive menu-driven interface.

This project is built using the four fundamental pillars of Object-Oriented Programming: Encapsulation, Inheritance, Polymorphism, and Abstraction. It demonstrates how real-world business problems can be modelled and solved using clean OOP design.

2. Project Objectives

1. Design a scalable hotel room management system using Java OOP principles.
2. Implement an abstract Room class with concrete subclasses for different room types.
3. Maintain a dynamic list of active bookings using Array List.
4. Generate accurate bills via a Bill interface.
5. Prevent double-booking of rooms through runtime validation.

3. Project Scope

The system covers the following functional areas:

- Guest check-in with personal details and room assignment
- Support for three room types: Single, Couple, and Suite
- Bill generation based on room rate and number of nights.
- View all active guest records.
- Guest check-out with automatic room release

4. OOP Concepts Used

OOP Concept	Implementation in Project
Abstraction	Abstract Room class with abstract get Price () and serotype () methods
Inheritance	Single Room, Couple Room, and Suite Room inherit from the abstract Room class
Polymorphism	Runtime polymorphism — room reference points to different subclass objects; display Room () overrides
Encapsulation	Private fields in Room and Guest classes accessed through public getters and setters

Interface	Bill interface implemented by Booking to enforce generate Bill () contract
Collections	Array List<Booking> stores and manages multiple active bookings dynamically

5. System Design & Architecture

5.1 Classes Overview

Class / Interface	Type	Responsibility
Room	Abstract Class	Base class for all room types; room number, booking status
Single Room	Concrete Class	Extends Room; price \$100/night
Couple Room	Concrete Class	Extends Room; price \$200/night
Suite Room	Concrete Class	Extends Room; price \$500/night
Guest	POJO Class	Stores' guest details: name, CNIC, phone, address
Bill	Interface	Contract declaring generate Bill () method
Booking	Concrete Class	Implements Bill; links Guest + Room + nights; computes bill
Main	Driver Class	Entry point; provides menu and controls program flow

5.2 Room Pricing

Room Type	Price per Night
Single Room	\$100
Couple Room	\$200
Suite Room	\$500

6. Features & Functionality

6.1 Check In

The system prompts the receptionist to enter by the guest's name, CNIC, phone number, and address. The user then selects a room number and number of nights and chooses a room type. If the room is already booked, the system alerts the user and prevents double-booking.

6.2 Generate Bill

By entering a room number, the system retrieves the guest's full details and calculates the total bill as: $\text{Total Bill} = \text{Room Price} \times \text{Number of Nights}$.

6.3 View Guest Details

Displays all currently checked-in guests with their room numbers, room types, and booking status.

6.4 Check Out

Locates the booking by room number, marks the room as available, removes the booking from the list, and confirms the checkout.

7. Tools & Technologies

- Language: Java (JDK 8 or above)
- IDE: IntelliJ IDEA / Eclipse / VS Code
- Data Structure: Array List for dynamic booking storage
- I/O: Scanner class for console-based input

8. Conclusion

The Smart Hotel Management System effectively demonstrates the application of core OOP concepts in a practical, real-world scenario. The project models hotel operations through a clean class hierarchy, enforces contracts via interfaces, and provides a fully functional interactive system for hotel room management.

This project serves as a strong foundation for further enhancement, such as adding a graphical user interface, database persistence, or online reservation support.